

FTRIA-101 — Runtime Invariant Discovery Algebra

Part II — Runtime Invariant Algebra

Sizhe Tan

with chatGPT (OpenAI)

2026-07-15

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

Part I established that Runtime Invariants exist within structured Configuration Spaces governed by Degrees of Freedom, admissible variations, and Constraint Surfaces.

The next question naturally follows:

How are Runtime Invariants discovered?

This paper introduces **Runtime Invariant Discovery Algebra (RIDA)** as the first branch of Runtime Invariant Algebra.

Rather than treating individual algorithms as unrelated engineering techniques, RIDA interprets them as different operators for discovering, approximating, refining, or localizing Runtime Invariants.

Under this perspective,

Common Concept Core (CCC),

Transformer embeddings,

Attention mechanisms,

representation learning,

graph matching,

feature extraction,

and numerous classical AI algorithms become members of a common Runtime Invariant Discovery family.

The objective of RIDA is therefore not to replace existing algorithms, but to provide a unified structural language through which they may be understood, compared, and connected.

1. From Algorithms to Runtime Functions

Traditional AI literature is typically organized according to algorithms.

Examples include

- Transformer,

- CNN,
- Graph Neural Networks,
- Reinforcement Learning,
- Decision Trees,
- Clustering,
- Representation Learning.

Although these algorithms appear fundamentally different, many perform remarkably similar structural functions.

FTRIA proposes organizing algorithms according to their **Runtime Function** rather than their implementation.

2. Runtime Invariant Discovery

The primary objective of Runtime Invariant Discovery is to identify stable structural entities that remain valid across multiple observations, representations, or runtime realizations.

Conceptually,
Observations

↓

Structural Regularities

↓

Runtime Invariant

↓

Executable Runtime Knowledge

The Runtime Invariant represents the stable structural identity extracted from varying implementations or observations.

3. Runtime Invariant Discovery Operators

Different algorithms employ different Discovery Operators.

Examples include

- Common Concept Core (CCC),
- Transformer embeddings,
- Attention,
- Graph matching,

- Representation learning,
- Feature extraction,
- Clustering,
- Differential Trees,
- XX Starmap,
- Contrastive learning.

Although their implementations differ,
their structural objective is similar:
Discover stable Runtime Invariants.

4. Explicit and Implicit Discovery

Runtime Invariant Discovery may occur in two different forms.

Explicit Discovery

The Runtime Invariant is directly represented and manipulated.

Examples include

- CCC,
- Differential Trees,
- Structural graphs,
- Runtime Invariant models.

Implicit Discovery

The Runtime Invariant is encoded indirectly inside learned representations.

Examples include

- Transformer embeddings,
- Attention,
- latent representations,
- neural feature spaces.

Both approaches pursue the same structural objective while using different computational mechanisms.

5. Discovery Rather Than Memorization

Runtime Invariant Discovery should not be confused with memorization.

Memorization preserves observations.

Discovery extracts structural regularities that survive many observations.

Consequently,

Discovery transforms data into reusable Runtime Knowledge.

This distinction explains why systems capable of Runtime Invariant Discovery often generalize far beyond individual training examples.

6. Discovery Across Modern AI

Runtime Invariant Discovery provides a common interpretation for diverse AI

paradigms.

Examples include

Structural Intelligence

- CCC,
- Differential Trees,
- XX Starmap,
- Runtime Invariants.

Transformer-based AI

- Word embeddings,
- Attention,
- Representation learning,
- Foundation models.

Vision AI

- Feature extraction,
- Visual representations,
- Multi-scale feature pyramids.

Graph AI

- Graph embeddings,
- Graph matching,
- Structural pooling.

Although developed independently,
these methods all perform variations of Runtime Invariant Discovery.

7. Runtime Invariant Discovery Algebra

The purpose of RIDA is not to invent another AI algorithm.

Instead,

it provides a higher-level algebra describing the discovery process itself.

Within this algebra,

different algorithms become different Discovery Operators acting upon
Runtime Invariant Configuration Spaces.

This interpretation enables meaningful comparison among previously
unrelated approaches.

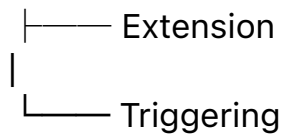
It also creates opportunities for transferring ideas across AI paradigms.

8. Relationship to the Remaining Runtime Algebra

Runtime Invariant Algebra consists of three complementary branches.

Runtime Invariant Algebra

|—— Discovery
|



Discovery identifies Runtime Invariants.

Extension expands or transforms Runtime Invariants while preserving identity.

Triggering activates Runtime Invariants during runtime execution.

Together, these three branches provide a unified interpretation of both existing and future AI systems.

Key Takeaways

- Runtime Invariant Discovery extracts stable structural identities from varying observations.
- Existing AI algorithms may be interpreted according to their Runtime Function rather than their implementation.
- Discovery may be explicit or implicit.
- CCC, Transformer embeddings, representation learning, graph matching, and feature extraction can all be viewed as Runtime Invariant Discovery operators.
- Runtime Invariant Discovery Algebra provides a common structural language rather than another competing algorithm.
- Discovery forms the first branch of the broader Runtime Invariant Algebra alongside Extension and Triggering.

Related FTRIA Documents

Part I — Foundations of Runtime-Invariant Degrees of Freedom

- FTRIA-001 — Runtime Invariant Degrees of Freedom
- FTRIA-002 — RI-Preserving Transformation
- FTRIA-003 — Admissible Runtime Variation
- FTRIA-004 — Constraint Surfaces and Invariant Boundaries
- FTRIA-005 — Local and Global Degrees of Freedom
- FTRIA-006 — Coupled Degrees of Freedom
- FTRIA-007 — Runtime-Invariant Configuration Space

Part II — Runtime Invariant Algebra

- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding

- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery
- FTRIA-110 — Runtime Invariant Extension Algebra
- FTRIA-120 — Runtime Invariant Trigger Algebra